What we knew till 2017

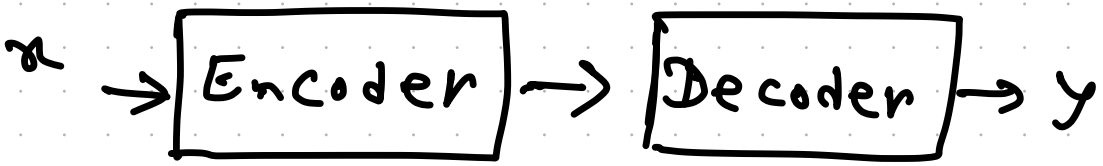
- RNNs are slow at inference: Sequential nature
- Gradients and history are tricky to handle for long sequences.
- Encoder - Decoder architecture is useful.
- Multi-step learning is better
- Residual connections are helpful: Vision domain ResNet
- Core idea of Vaswani et al.

Why not combine all good things and get rid of the problematic parts?

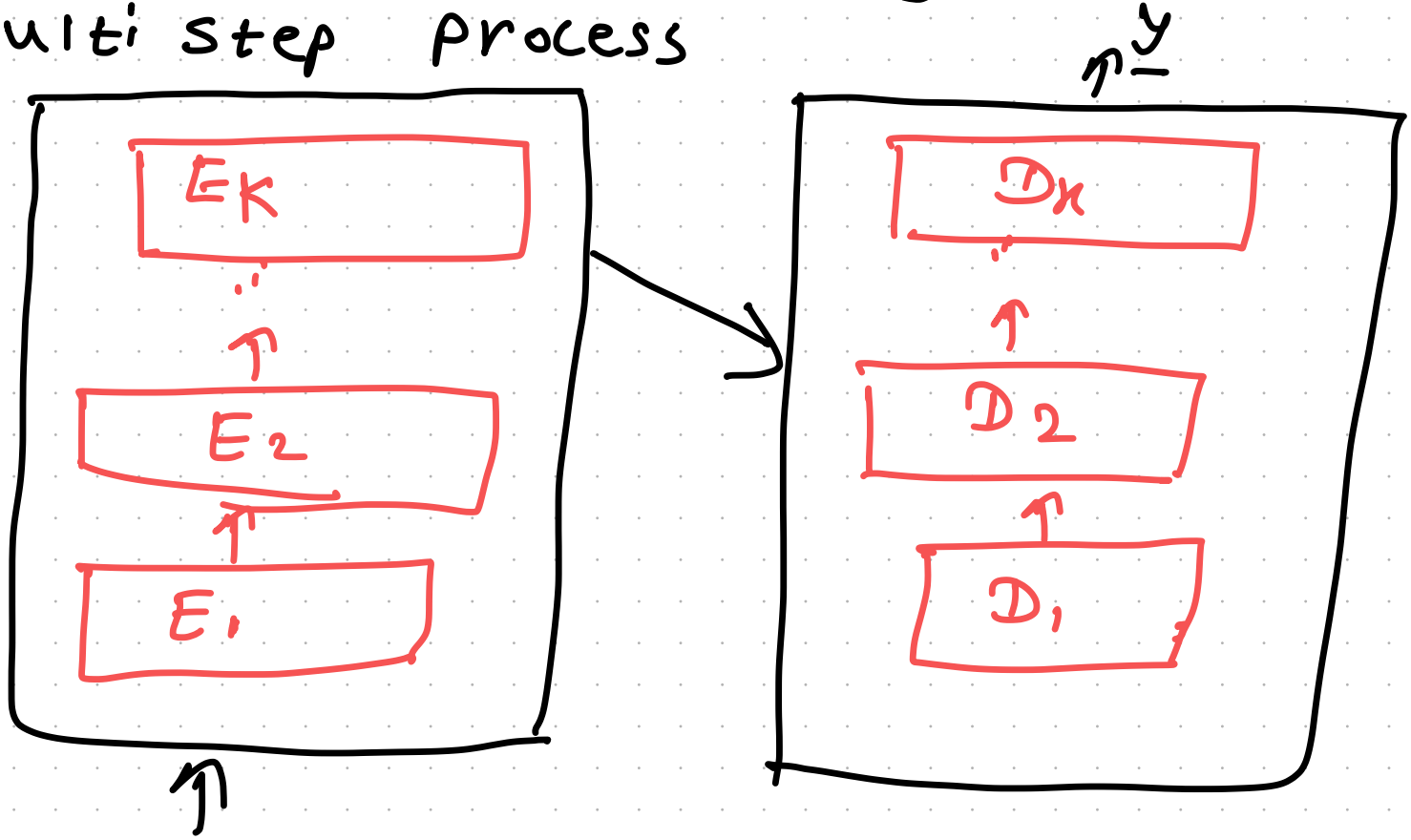
Core Concepts of Transformer

- Self attention
- Multiple heads
- Masked attention
- Encoder-Decoder attention
- Residual connections
- Positional embeddings

High Level Architecture



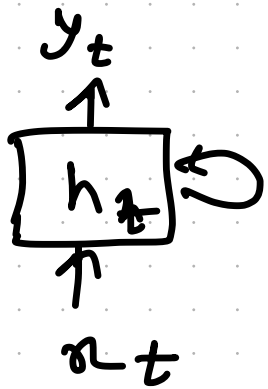
Let us make encoding and decoding multi step process



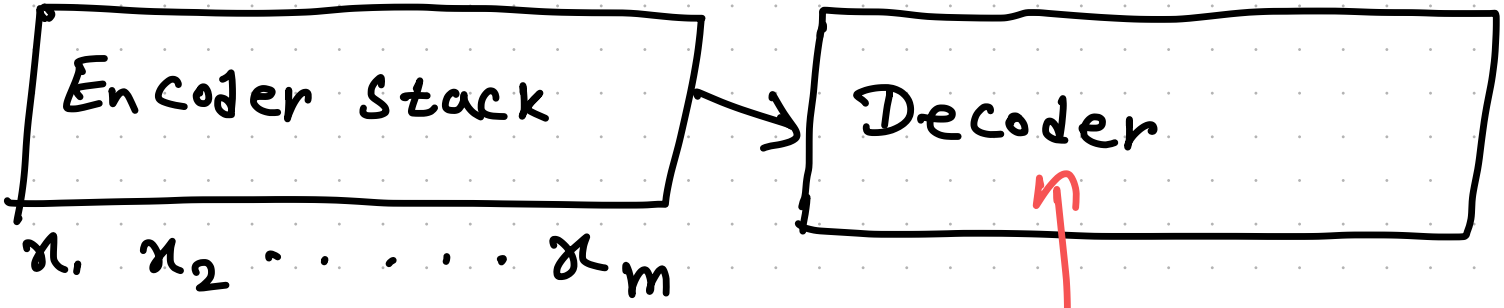
- All E_s are identical. All D_s are identical.

No Recurrence (on input)

RNN:



Transformer

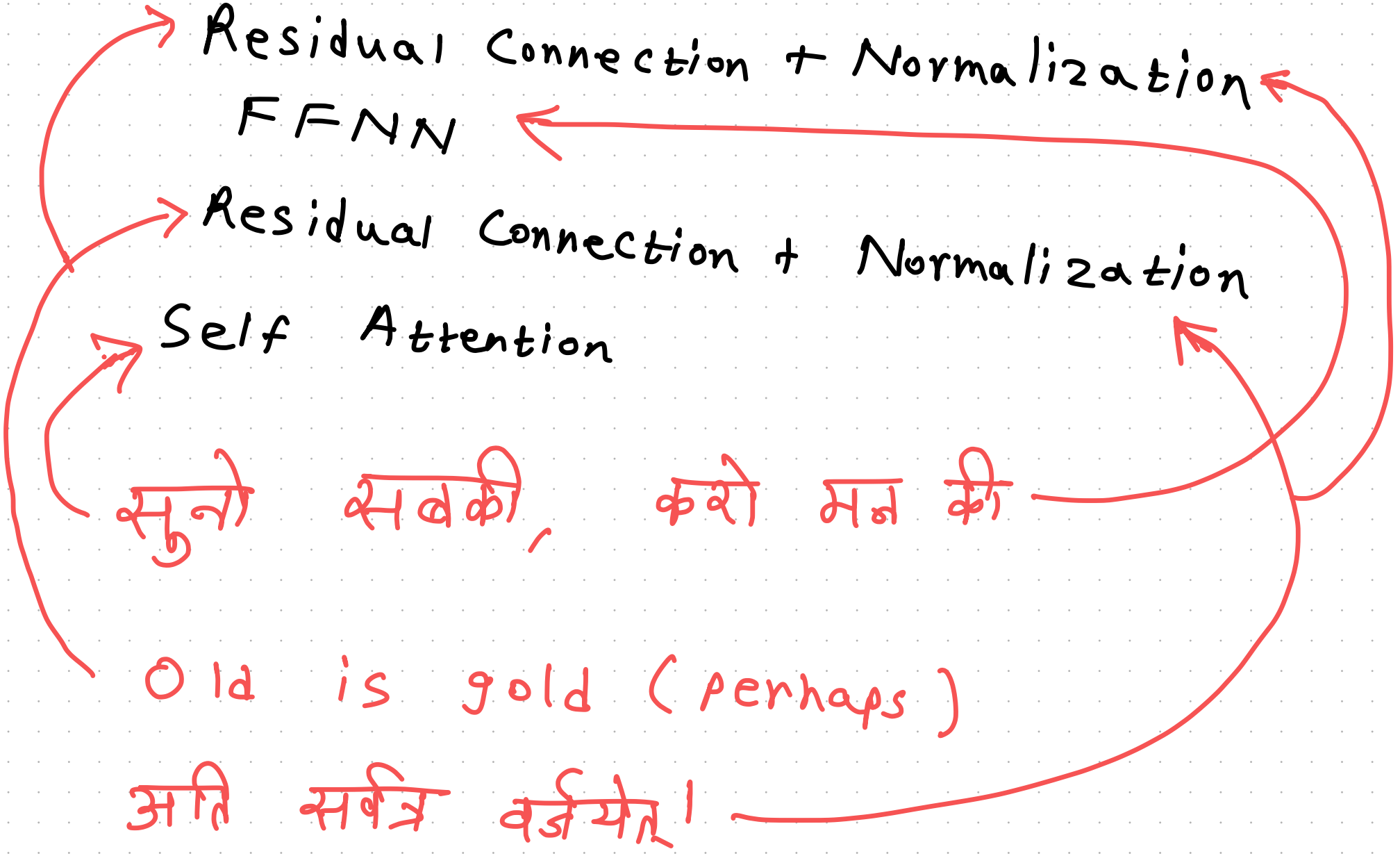


Add position embedding to keep track of order

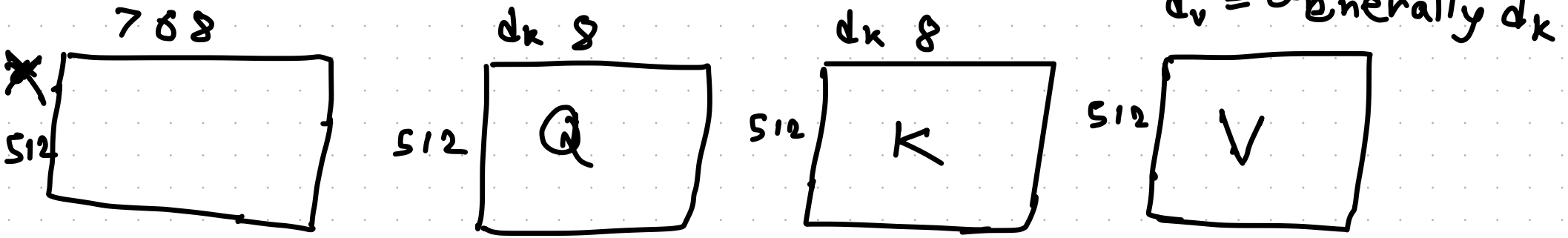
Still recurrent!

Do NOT forget that

Encoder Architecture



Self Attention Block



Now each token has three representation

$q_i = x_i W_Q$ $\quad k_i = x_i W_K$ $\quad v_i = x_i W_V$

768×8

x_i = row of X

q_i — Q

K_i — K

V_i — V

Output representation

$$Z_i = \sum_{K=1}^{512} W_{iK} V_K$$

$$W_{iK} \propto q_i \cdot K_K$$

$$Attention(Q\,K\,V) = Softmax\left(\frac{Q K^T}{\sqrt{d_k}}\right) V$$

Scaling for stability

There are multiple attention heads

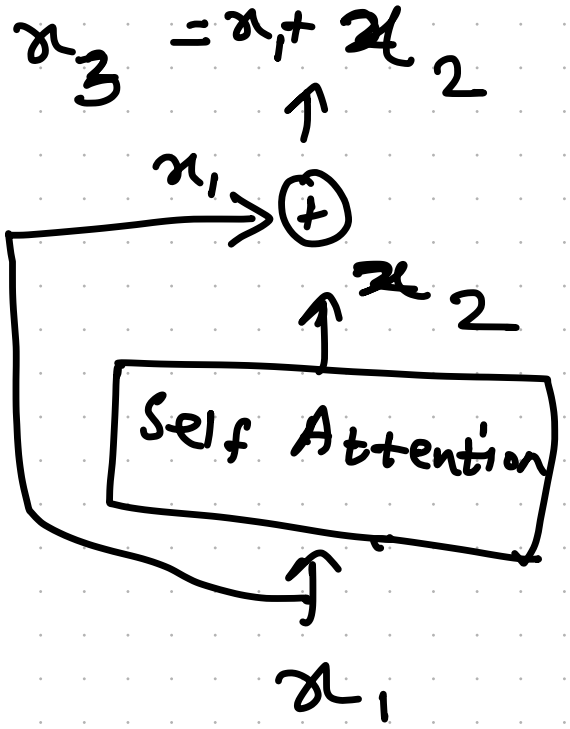
- Final output from self-attention block

$$\text{Multihead}(Q, K, V) =$$

$$\text{Concat}(\text{Head}_1, \text{Head}_2 \dots \text{Head}_h) \cdot W^0$$

$$W^0 : ((h \times d_v) \times d_{model})$$

Residual Connection



- Deal with Vanishing gradient problem

$$\frac{\partial E}{\partial w_1} = \frac{\partial E}{\partial w_n} \cdot \frac{\partial w_n}{\partial w_{n-1}} \cdots \cdots \frac{\partial w_2}{\partial w_1}$$

Vanishing gradient

- Weight magnitude
- Activation gradient

- Self attention might output 2 as zeros.
- Sometimes old representation still carries the value.
- Residual connection is the hedge against wrong or bad transformation.

Layer Norm

$x_3^0 \; x_3^1 \cdots x_3^{512}$: Vector for a single token

$$x_4^i = \frac{x_3^i - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

mean across all dimensions of x_3

avoid division by zero

variance of x_3 across all dimensions

$$x_5^i = \gamma_a^i x_4^i + \beta_a^i$$

Learnable parameters

- Numerical Stability
- Avoid exploding and vanishing gradients problem
- Mean zero, allows to use both sides of activation
- Variance one, keeps magnitude in check

Feed Forward Layer

- Single hidden layer with ReLU activation

$$x_6 = \left(\text{ReLU}\left(x_5 W_1 + b_1\right)\right) W_2 + b_2$$

You can use any other activation function

- Large hidden size: $512 \xrightarrow{d_{ff}} 2048 \rightarrow 512$
- One more residual connection

$$x_7 = x_5 + x_6$$

- One more layer norm

$$x_8^i = \frac{x_7^i - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

$$x_9^i = \gamma_b^i \, x_8^i + \beta_b^i$$

Learnable parameters

Without Position Encoding, it's a Bag of Words

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{(2i/d_{model})}}\right)$$

$$PE(pos, 2i+1) = \cos\left(\quad\right)$$

- What happens if same word appears at multiple positions?
- $x_1 = x_0 + PE(pos)$
- $PE(pos)$ is a distinct vector for each position
- Position encoding is added only at the first encoder. Rest of the encoders do not use it explicitly.

Encoder Stack has multiple encoders

- Multi step transformation has higher chance of being successful.
- Let us do the encoder side parameter count

V = vocab size d_{model} = Model dimensionality

N = Number of tokens h = Number of heads

$d_k = \frac{d_{model}}{h}$ d_{ff} = FFN hidden layer size

Base model

$V = 37K$ $d_{model} = 512$

Total ≈ 3.15 M per encoder

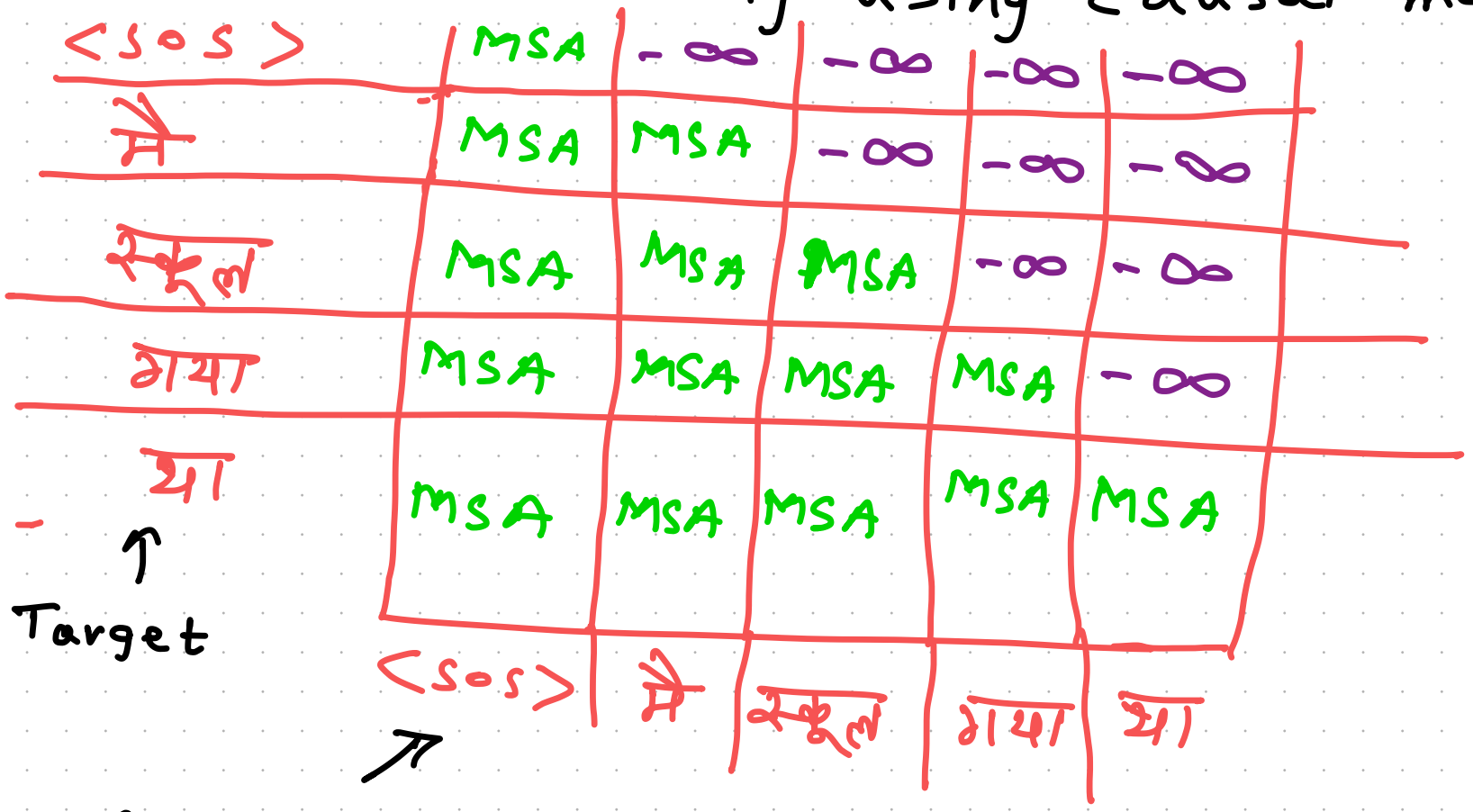
- $W_Q\ W_K\ W_V\ W_O$: $4 \times d_{model}^2 \approx 1.04$ M
  $+ 4 \times d_{model}$ bias $\approx 2K$
- FFN: $(512 \times 2048) + (512 \times 2048)$ + biases ≈ 2.09 M
- Layer Norm: $2 \times 2 \times d_{model} \approx 2K$

Decoder Work Sequentially Like an RNN during the inference

- Using previously predicted sequence, predict next token
  - Assume first fake input <SOS>
  - Teacher forcing during training
  - Translation task
    I went to school → मै स्कूल गया था
- At Decoder
  <SOS> → मै
  <SOS> मै → स्कूल
  <SOS> मै स्कूल → गया
  <SOS> मै स्कूल गया → था
  <SOS> मै स्कूल गया था → <EOS>

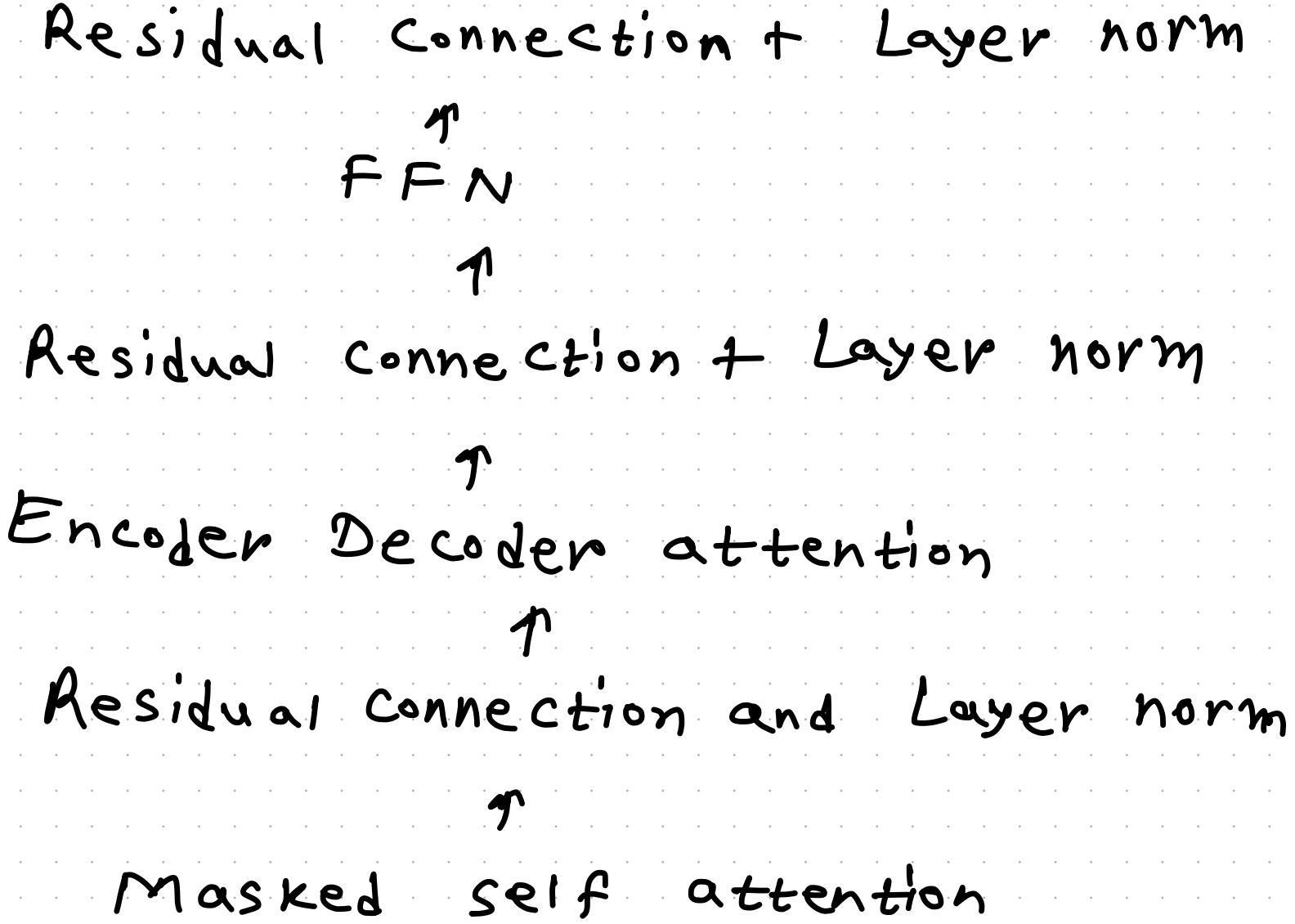
During training decoder learns all prefixes in parallel

- Teacher forcing
- Faster training using causal mask

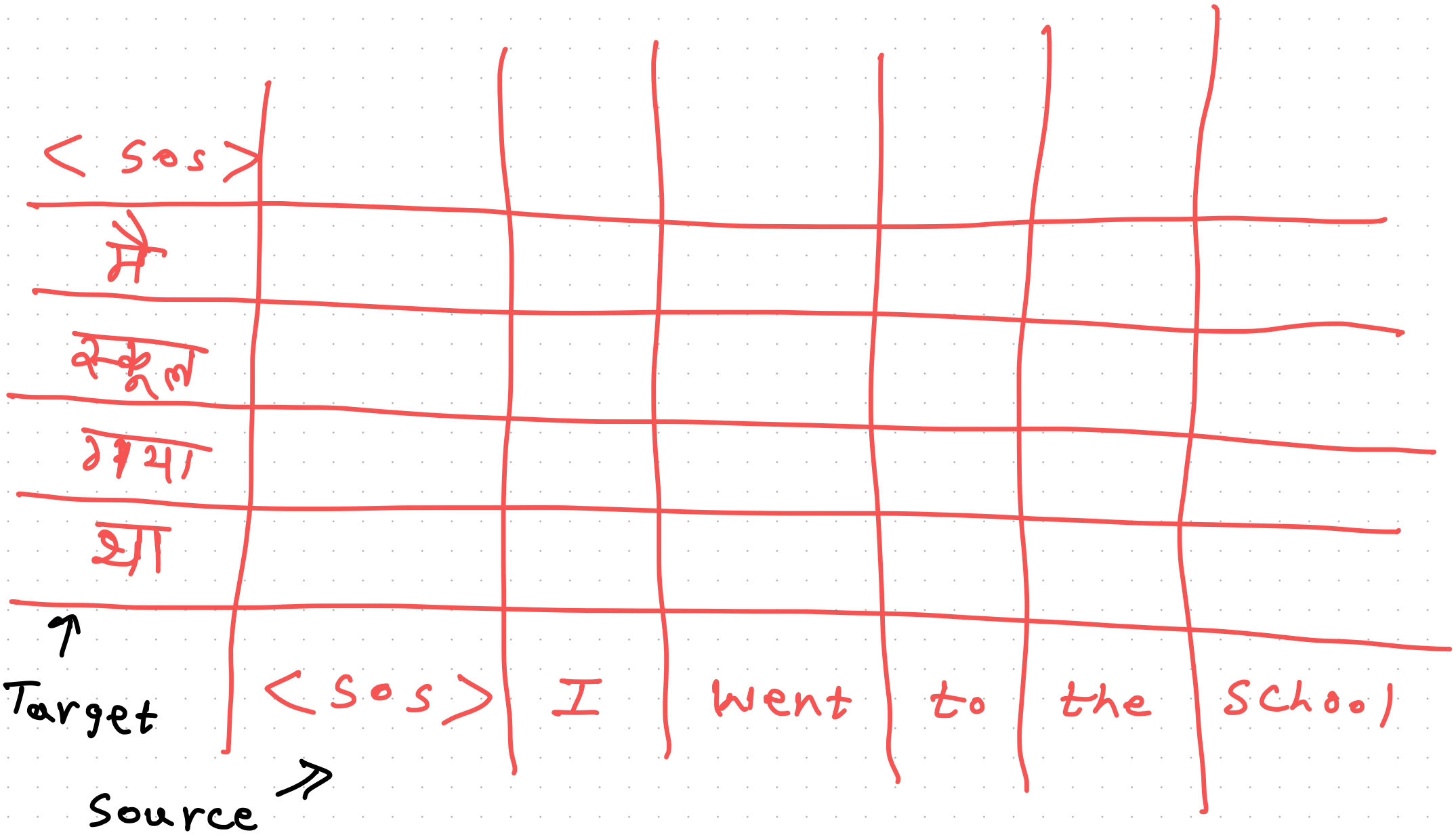


Target attends to the Source.

Decoder Architecture



Encoder Decoder Attention



Target attends to the source.

Q K V in Encoder Decoder Attention

- Q representation of all target words
- K, V representation of all Source words
- New representation of each target word

$$r_t = \sum_{j=1}^{\text{source length}} w_{jt} V_j$$

$$w_{jt} \propto q_t \cdot k_j$$

Using the last decoder's last word representation, we predict the next word

- r_t $1 \times d_{model}$ W_{out} $d_{model} \times V_{decoder}$ $z_{1 \times V_{decoder}}$
- softmax(z) gives us the probability distribution
- At training time, the whole sequence is predicted in one go for efficiency.

Decoder Parameter Count

Masked Self Attention: W_Q W_K W_V W_O

$(512 \times 512 \times 4) + (512 \times 4)$

↑ weights

↑ biases